

コマ 3: 算術式コード生成

今日のゴール

整数リテラルと算術演算を RV64 アセンブリに変換する。

第 02 回では、AST を直接評価して Python の整数値を返した。この回では、AST を評価する代わりに、同じ計算を行う RV64 アセンブリを出力する。

`codegen(node)` の約束は次の通り。

`node` が表す式を計算し、その結果が実行時に `a0` レジスタに入るようなアセンブリを出力する。

たとえば、`return 1 + 2 * 3;` をコンパイルし、生成されたプログラムを実行すると、終了コードが 7 になることを目指す。

1 この回で扱う範囲

対象にするプログラムは、次の形に限定する。

```
int main() {  
    return 式;  
}
```

扱う式は以下の通り。

種類	例
整数リテラル	42
単項マイナス	-3
足し算	1 + 2
引き算	10 - 3
掛け算	2 * 3
割り算	10 / 2
剰余	10 % 3

種類	例
カッコ	(1 + 2) * 3

変数、代入、if、while、関数呼び出しはまだ扱わない。

2 ASTを確認する

第 02 回と同じように、まず Parser が返す AST を確認する。

```
python3 scaffold/parse_viewer.py sessions/03_arithmetic_codegen/tests/complex.c
```

このプログラムの内容は次の通り。

```
int main() {
    return (100 - 3 * 7) / 4 + 1;
}
```

実行すると、次のような S 式が表示される。

```
(program
  (funcdef "main" :type int (params)
    (block
      (return
        (add
          (div
            (sub (num 100)
              (mul (num 3) (num 7)))
            (num 4))
          (num 1))))))
```

この式は、元の C コードでは次の形である。

```
return (100 - 3 * 7) / 4 + 1;
```

Parser は演算子の優先順位とカッコを処理し、次のような構造の AST を返している。

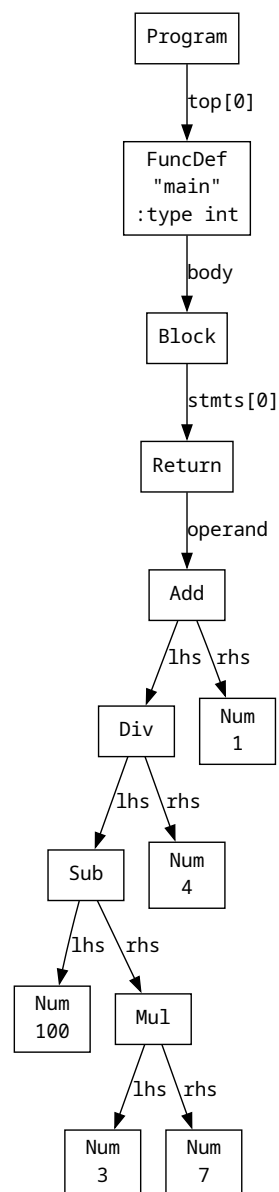


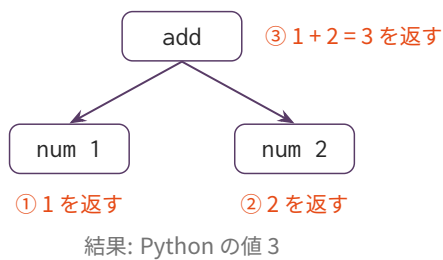
図 1 `complex.c` の AST

```

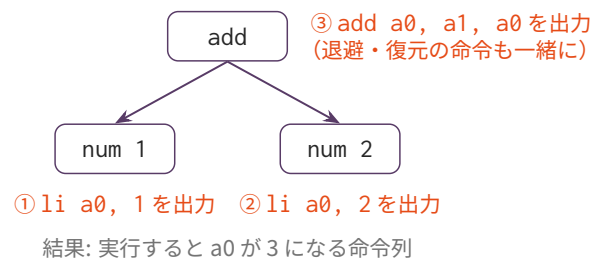
(add
  (div
    (sub (num 100)
      (mul (num 3) (num 7)))
    (num 4))
  (num 1))
    
```

`codegen()` は、この木を再帰的にたどりながら、各ノードに対応するアセンブリを出力する。

コマ 2 eval_ast — 値を返す



コマ 3 codegen — 命令を出力する



木のたどり方は同じ。「値を返す」部分を「命令を出力する」に置き換えただけ

図 2 eval_ast (値を返す) と codegen (命令を出力する) の対比

3 第 02 回との違い

第 02 回の eval_ast() は、計算結果を Python の値として返していた。

```
eval_ast(node) -> 整数値
```

第 03 回の codegen() は、計算結果を直接返さない。代わりに、計算を行うアセンブリを出力する。

```
codegen(node) -> アセンブリを出力する
```

ただし、再帰的に AST をたどる考え方は同じである。

第 02 回

第 03 回

ND_NUM を見たら数値を返す

ND_NUM を見たら li を出力する

ND_ADD なら左右を評価して足す

ND_ADD なら左右を計算する命令列を出力して add する

結果は Python の戻り値

結果は実行時の a0 レジスタ

木のたどり方は第 02 回と同じで、「値を返す」部分が「命令を出力する」に置き換わる。

4 使う RV64 命令

この回で使う主な命令は以下である。

命令	意味
<code>li a0, N</code>	即値 <code>N</code> を <code>a0</code> に入れる
<code>neg a0, a0</code>	<code>a0</code> の符号を反転する
<code>add a0, a1, a0</code>	<code>a1 + a0</code> を <code>a0</code> に入れる
<code>sub a0, a1, a0</code>	<code>a1 - a0</code> を <code>a0</code> に入れる
<code>mul a0, a1, a0</code>	<code>a1 * a0</code> を <code>a0</code> に入れる
<code>div a0, a1, a0</code>	<code>a1 / a0</code> を <code>a0</code> に入れる
<code>rem a0, a1, a0</code>	<code>a1 % a0</code> を <code>a0</code> に入れる
<code>addi sp, sp, -8</code>	スタックを 8 バイト分確保する
<code>sd a0, 0(sp)</code>	<code>a0</code> の値をスタックに保存する
<code>ld a1, 0(sp)</code>	スタックから値を読み、 <code>a1</code> に入れる
<code>addi sp, sp, 8</code>	確保したスタックを戻す

5 codegen の基本方針

`CodeGen03` クラスの `codegen(node)` は、`node.kind` を文字列で見て `match` で処理を分ける。本体のディスパッチはスケルトンにすでに書いてある。学生が実装するのは各 `handler method` である。

<code>node.kind</code>	実装する handler method	生成する処理
<code>'Num'</code>	<code>codegen_Num</code>	<code>node.val</code> を <code>a0</code> に入れる
<code>'Neg'</code>	<code>codegen_Neg</code>	<code>node.operand</code> を計算し、 <code>a0</code> を <code>neg</code> する

node.kind	実装する handler method	生成する処理
'Add'	codegen_Add	左右を計算し、足す
'Sub'	codegen_Sub	左右を計算し、引く
'Mul'	codegen_Mul	左右を計算し、掛ける
'Div'	codegen_Div	左右を計算し、割る
'Mod'	codegen_Mod	左右を計算し、剰余を求める

重要な約束は、どのノードでも、`codegen(node)` が終わった時点でそのノードの計算結果が `a0` に入っているようにすることである。

二項演算の handler は共通補助メソッド `_codegen_binary_value(node, op)` に委譲するとコードの重複が減らせる。

6 二項演算の生成パターン

二項演算では、左辺と右辺の 2 つの値が必要になる。

しかし、`codegen()` の結果は常に `a0` に入る。そのため、左辺を計算した後、その値を一度スタックに退避してから右辺を計算する。

基本パターンは次の通り。

1. 左辺を `codegen` する
-> 左辺の値が `a0` に入る
2. `a0` をスタックに保存する
3. 右辺を `codegen` する
-> 右辺の値が `a0` に入る
4. スタックから左辺の値を `a1` に戻す
5. `a1 op a0` を計算し、結果を `a0` に入れる

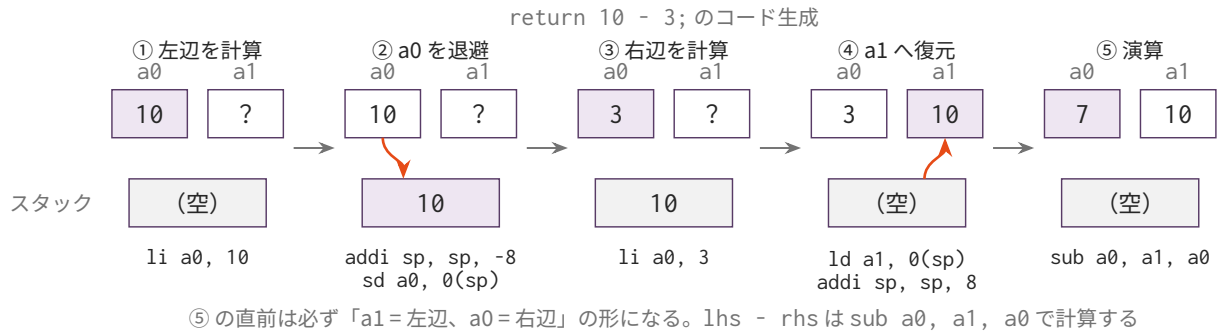


図3 return 10 - 3; を生成する 5 ステップと、レジスタ・スタックの変化

この退避のパターンは、この後の代入（コマ 4）や関数呼び出し（コマ 8）でも繰り返し使う。

このパターンを使うと、二項演算の直前には次の状態になる。

レジスタ	内容
a1	左辺の値
a0	右辺の値

したがって、10 - 3 は次のように計算する。

```
sub a0, a1, a0
```

これは a0 = a1 - a0、つまり「左辺 - 右辺」を意味する。

div と rem も順序が重要である。

C の式	使う命令
lhs - rhs	sub a0, a1, a0
lhs / rhs	div a0, a1, a0
lhs % rhs	rem a0, a1, a0

sub a0, a0, a1 のように逆にすると、rhs - lhs になってしまう。

7 コード生成器の枠組み

コード生成器は `Codegen03` クラスとして実装する。すべての状態はインスタンス変数に保持され、アセンブリの行は `list[str]` に蓄積して最後にまとめて出力する。

```
class Codegen03:
    def __init__(self) -> None:
        self._out: list[str] = []    # 出力行バッファ

    def emit(self, line: str) -> None:
        self._out.append(line)

    def output(self) -> str:
        return '\n'.join(self._out) + '\n'
```

`gen_func(node)` は関数定義ノードを受け取り、次の `hook` を順に呼んでアセンブリを出力する。

- | | |
|---|---|
| 1. <code>_reset_func_state(node)</code> | → 関数状態の初期化（コマ 3 では空） |
| 2. <code>_emit_func_prologue(name)</code> | → プロローグを出力する |
| 3. <code>_emit_func_body(body)</code> | → <code>main</code> の <code>return</code> 式を <code>codegen()</code> に渡す |
| 4. <code>_emit_func_epilogue()</code> | → エピローグを出力する |

コマ 3 では関数の枠組み（プロローグ・エピローグ）も学生が実装する。後続回では一度作ったこの枠組みが `importlib` による継承で引き継がれる。

この回のプロローグは、最低限のスタックフレームを作るためのものである。

```
addi sp, sp, -16
sd ra, 8(sp)
sd s0, 0(sp)
addi s0, sp, 16
```

各レジスタの意味は次の通り。

レジスタ	役割
------	----

<code>a0</code>	戻り値を入れるレジスタ
<code>sp</code>	スタックポインタ
<code>s0</code>	フレームポインタ

レジスタ	役割
------	----

ra	関数から戻る先のアドレス
----	--------------

この回では、ローカル変数や関数呼び出しはまだ扱わない。そのため、プロローグ・エピローグの細部を完全に理解していなくても、`codegen()` の実装に進んでよい。

ただし、最終的に `ret` する時点で `a0` に `return` 式の値が入っていれば、C の `main` 関数の戻り値として扱われる。`qemu-riscv64 ./out; echo $?` で確認している終了コードは、この `a0` の値である。

8 編集するファイル

- `mycc.py`

`Codegen03` クラスの以下の TODO を埋める。

実装対象	役割
<code>codegen_Num</code>	既に実装例がある。 <code>li a0, node.val</code>
<code>codegen_Neg</code>	単項マイナス
<code>codegen_Add</code> / <code>codegen_Sub</code> / <code>codegen_Mul</code> / <code>codegen_Div</code> / <code>codegen_Mod</code>	二項演算。 <code>_codegen_binary_value</code> に委譲
<code>_codegen_binary_value</code>	二項演算の共通処理（退避 → 計算 → 復元）
<code>_emit_func_prologue</code>	<code>.globl</code> , 関数ラベル, <code>ra/s0</code> 保存
<code>_emit_func_body</code>	<code>body.stmts</code> を <code>gen_stmt()</code> で処理
<code>_emit_func_epilogue</code>	<code>s0/ra</code> 復元、 <code>sp</code> を戻し <code>ret</code>

実装対象	役割
<code>gen_stmt_Return</code>	<code>return</code> の式を <code>codegen()</code> し、結果を <code>a0</code> に残す

`codegen(node)` 本体の `match` ディスパッチはスケルトンにあらかじめ書かれている。

9 tests/

ファイル	内容	期待値
<code>target.c</code>	<code>1 + 2 * 3</code>	7
<code>prec_div.c</code>	<code>(100 - 3 * 7) / 4 + 1</code>	20
<code>mod_div.c</code>	<code>100 / 4 + 1</code>	26
<code>nested.c</code>	<code>(1 + 2) * (3 + 4)</code>	21

```
add.c | 1 + 2 | 3 |
sub.c | 100 - 79 | 21 |
mul.c | 掛け算 | 対応する .ans を参照 |
div.c | 17 / 5 | 3 |
mod.c | 17 % 5 | 2 |
prec.c | 2 + 3 * 4 | 14 |
paren.c | カッコ付き算術式 | 対応する .ans を参照 |
complex.c | (100 - 3 * 7) / 4 + 1 | 20 |
```

10 テスト

```
python3 scaffold/test_runner.py sessions/03_arithmetic_codegen
```

このテストは次の流れで動く。

C ソース

- > mycc.py が RV64 アセンブリを出力
- > riscv64-linux-gnu-gcc が実行ファイルに変換
- > qemu-riscv64 で実行
- > 終了コードを .ans と比較

個別に動かす場合は、次のようにする。

```
python3 sessions/03_arithmetic_codegen/mycc.py sessions/03_arithmetic_codegen/test
| riscv64-linux-gnu-gcc -x assembler -static - -o out

qemu-riscv64 ./out
echo $?
```

終了コードが 7 になれば、基本形は成功である。

11 注意

この回のテストでは、割り算・剰余の右辺は 0 にならない。

また、この回では負数を含む割り算・剰余は扱わない。Python の `//` や `%` と同様に、言語や実行環境によって負数の扱いが混乱しやすいため、まずは非負整数の式に限定する。